```
agent Researcher(topic: text) -> text {
  goal "Find five facts about {topic}"
  policy { require human when confidence < 0.70 }
}
flow main(topic: text) {
  [a, b] = par { Writer(t), Critic(t) }
  emit a, b
}
```

OPEN-SOURCE PROGRAMMING LANGUAGE · v0.1

AlgoLang

An agent-native language for building AI systems

Every unit of work is an **agent**.



Dr. S. Satyanarayana Ph.D., PDF (AI)

Founder, CEO & Chief Agentic AI Scientist · AlgoProfessor AI R&D Solutions, Hyderabad

www.algoprofessor.com · ceo@algoprofessor.com

Agenda

1. **Why a new language?**
   the problem and the core idea
2. **The language**
   types · agents · policies · flows
3. **Runtime & safety**
   call lifecycle · audit log · ethics guard
4. **Three ways to work**
   command line · Studio GUI · AI writer
5. **Inside the compiler**
   pipeline · generated Python
6. **Comparison**
   vs Python, Java, Rust, Mojo, Jac, Zero
7. **Real applications**
   8 domains, working code
8. **Responsible AI & teaching**
   patterns · limitations · 10 labs
9. **Open-source project**
   roadmap · licence · get started

AlgoLang by the numbers

1
primitive for work: the **agent**

5
policy rules built into the syntax

3
ways to work: CLI · GUI · agentic

8
real application blueprints

5
types: text, score, list, json, bool

15
compile-time checks (E100–E114)

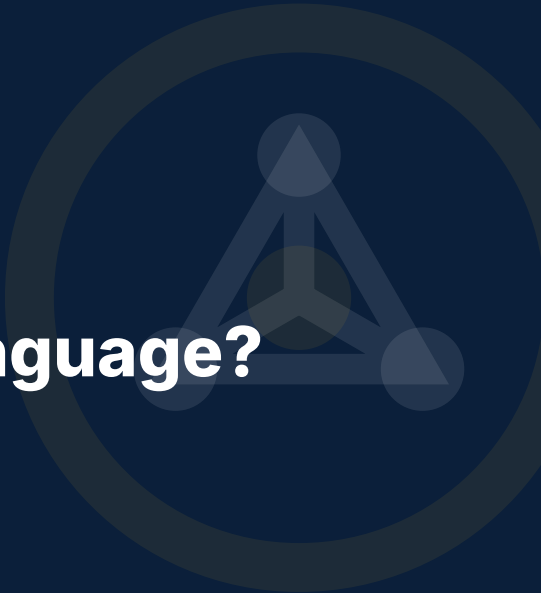
5
platforms: Windows, Linux, macOS, Docker, Colab

77
pages of official documentation

Toolchain: about 3,000 lines of Python and HTML — small enough for students to read end to end.

01

Why a new language?



The problem: AI work hides inside silent functions

In today's languages the unit of work is the **function**. When an AI model does the work inside it, nobody can easily answer:

1. **Who** decided — which worker, which model?
2. **What** was it asked to do?
3. With **how much confidence**?
4. Did a **human** check it?
5. Was a **protected attribute** (caste, gender) used?
6. What did it **cost**, and where is the **record**?

Where this matters

Admissions · hiring · credit · healthcare · peer review · exams — AI decisions that affect people need accountability *by design*, not by afterthought.

The AlgoLang answer

Make each of those six questions a **part of the language**, checked by the compiler and enforced by the runtime.

The core idea: agents, not functions



The AlgoLang rule

There are **no functions**. The only thing a program can call is an **agent**. A **flow** orchestrates agents and nothing else.

goal	what was it asked to do?
role	in what capacity?
uses	which tools may it touch?
model	which model answered?
policy	cost, retries, human review, forbidden inputs
audit	what happened, when?

```
agent Reviewer(draft: text) -> score {
    role   "Peer reviewer"
    goal   "Score {draft} from 0 to 1 for accuracy"
    uses   web.search
    model  smart
    policy {
        budget tokens 4000
        retry 2
        require human when confidence < 0.70
        forbid gender, caste, religion
    }
}
```

```
E101 'summarise_text' is not an agent —
     in AlgoLang every unit of work is an agent
```

AlgoLang at a glance

```
model fast = "claude-haiku-5-5"

agent Writer(facts: text, tone: text = "academic") -> text {
    goal  "Write a 200-word brief from {facts} in a {tone} tone"
    model fast
    policy { budget tokens 4000  retry 1 }
}

agent Reviewer(draft: text) -> score {
    goal   "Score {draft} from 0 to 1 for accuracy"
    policy { require human when confidence < 0.70 }
}

flow main(facts: text) {
    [formal, simple] = par { Writer(facts), Writer(facts, tone = "
      plain") }
    if Reviewer(formal) < 0.80 {
        formal = Writer(facts, tone = "rigorous academic")
    }
    emit formal, simple
}
```

- Two writers run **in parallel** (**par**)
- A reviewer **scores** the formal draft
- Low-confidence scores go to a **human**
- A low score triggers a **redraft**
- **Budgets** and **retries** on every writer
- Every call written to an **audit log**
- Everything **checked before it runs**

02

The language



A program has four kinds of declaration

model

Names a model ID; agents use the short name.

```
model smart =
"claude-sonnet-5-5"
```

tool

Declares a capability. Gives it to *no one* until an agent **uses** it.

```
tool web.search
```

agent

The only unit of work: typed inputs, a return type, a goal and a policy.

```
agent A(x: text)
-> score { …}
```

flow

Wires agents together. `flow main` is the entry point.

```
flow main(t: text)
{ …}
```

Lexical basics

`//` comments · braces, no significant whitespace · `;` optional · strings in `" "` · `{param}` placeholders inside goals, checked by the compiler (E109)

25 keywords, no operators for computing

Flows only *route* values; all computation is done by agents. Small enough to learn in one class.

Five types — and every result carries provenance

- **text** — free text — facts, drafts, letters
- **score** — number 0–1; the *only* comparable type
- **list** — ordered items — the input to **`par for`**
- **json** — structured records
- **bool** — pass / fail checks

Comparing text is a compile error:

```
if summary < 0.5 { …} → E110
```

Every agent call returns a Result

`value`	the answer, of the declared type
`confidence`	0–1, reported with the answer
`agent`	who produced it
`call_id`	link to its audit record
`status`	ok · escalated · approved

Why

Hidden uncertainty breeds over-trust; visible confidence makes it natural to route doubtful results to people.

Policies: governance as syntax

budget tokens N

Blocks a call *before* it is sent if the prompt is over budget; caps reply length in live mode.

run time

retry N · timeout S

Retries failed calls; treats slow calls as failures. Policy violations are never retried.

run time

require human when confidence < X

Low-confidence results become **escalated**; with `--interactive` a person approves or rejects.

run time

forbid gender, caste, …

Agents may not take these as inputs (E108); calls whose inputs mention them are **blocked**.

compile + run time

Policies travel with the agent.

A caller cannot forget them, and every audit record lists the exact rules that were in force for that call.

No unbounded loops.

Every program finishes, and its maximum number of agent calls — its maximum cost — can be read from the source.

Flows: five statements, nothing else

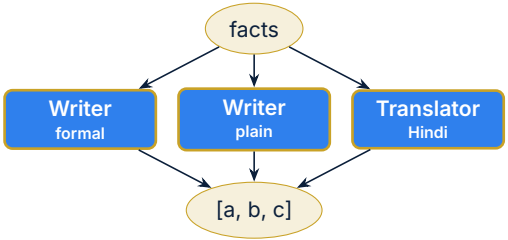
Call	`x = Agent(a, k = v)`
Parallel	`[a, b] = par { A(x), B(x) }`
Parallel map	`rs = par for c in xs { A(c) }`
Branch	`if s < 0.8 { …} else { …}`
Output	`emit a, b`

- Positional or keyword arguments; defaults may be omitted
- Conditions compare scores, numbers, booleans
- Flows **cannot be called** and cannot compute

```
flow main(paper: text) {
    criteria = Planner(paper)
    reports  = par for c in criteria {
                   CriterionReviewer(c, paper)
     }
    ethics   = EthicsAuditor(paper)
    verdict  = Editor(reports, ethics)
    if verdict >= 0.5 {
        letter = AcceptLetter(reports)
    } else {
        letter = RejectLetter(reports)
    }
    emit verdict, letter
}
```

Multi-agent parallelism is a keyword

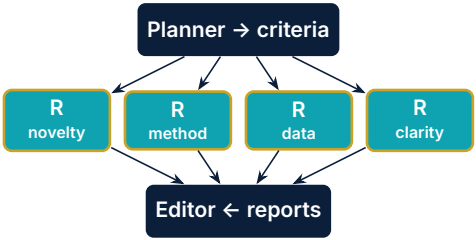
par { A, B, C }



Results keep their order

Names bind in the order written, whatever order agents finish.

par for c in criteria { R(c) }



Fan-out / fan-in in one line

Time is roughly that of the slowest worker, not the sum.

15 compile-time checks — before a single token is spent



- **E101** not an agent (the core rule)
- **E102** wrong arguments
- **E103** undefined variable
- **E100** syntax error
- **E107** defined twice
- **E104** undeclared tool
- **E106** unknown model
- **E108** forbidden attribute as input
- **E109** unknown {placeholder} in goal
- **E112** agent without a goal
- **E105** unknown type
- **E110** comparing a non-score
- **E111** no flow main
- **E113** [a, b] does not match par
- **E114** par for over a non-list

correctness · governance & capabilities · types & structure

All errors are reported together, with line numbers. A program with errors never runs.

The compiler at work

```
agent Summariser(doc: text) -> text {
    goal "Summarise {document}"
    uses email.send
    model turbo
}
agent Screener(caste: text) -> score {
    goal "Score applicant {caste}"
    policy { forbid caste }
}
flow main(doc: text) {
    s = Summariser(doc, doc)
    t = summarise_text(doc)
    if s < 0.5 { emit s }
    emit missing
}
```

```
$ algo check bad_example.algo
✗ bad_example.algo: 8 error(s)
  line 5: E109 agent Summariser: goal mentions {document}
          but there is no parameter 'document'
  line 5: E104 agent Summariser uses tool 'email.send',
          which is not declared (add `tool email.send`)
  line 5: E106 agent Summariser: unknown model 'turbo'
  line 12: E108 agent Screener takes forbidden attribute
          'caste' as an input
  line 19: E102 Summariser takes 1 argument(s) (doc), got 2
  line 20: E101 'summarise_text' is not an agent — in
          AlgoLang every unit of work is an agent
  line 21: E110 's' is text, not a score
  line 22: E103 undefined variable 'missing'
```

03

Runtime & safety



Every agent call passes through seven steps



Policy steps

Steps 3, 4 and 6 (purple) enforce the agent's policy on *every* call.

Concurrency

`par` uses a thread pool; output and audit writes never interleave.

Exit codes

0 ok · 1 compile/input error · 2 policy violation · 3 agent failure

Accountable by default: the audit log


```
{
  "call": "d5a1ba",
  "agent": "Researcher",
  "model": "claude-sonnet-5-5",
  "backend": "mock",
  "inputs_sha": "23cd4baf6ab210a5",
  "returns": "text",
  "confidence": 0.83,
  "tokens": 93,
  "status": "ok",
  "tools": ["web.search"],
  "policy": {"budget_tokens": 6000, "retry":
     2},
  "run": "832d55bb",
  "program": "research_brief",
  "ts": "2026-10-28T11:20:13"
}
```


- **One JSON line per agent call** — no logging code to write
- Records model, confidence, tokens, status, tools and the **policy in force**
- **Blocked** and **failed** calls recorded too, with the reason
- Loads straight into Excel or `pandas`

Accountability without over-collection

Stores a SHA-256 *hash* of the prompt, not the prompt itself — an auditor can verify an input without the log becoming a copy of sensitive data.

The ethics guard in action

```
agent Editor(reports: list, ethics: text)
      -> score {
  role  "Editor-in-Chief"
  goal  "Weigh {reports} and {ethics}"
  policy {
    require human when confidence < 0.75
    forbid gender, caste, religion,
           nationality
  }
}
```

What happened

The title mentions *nationality*; it reached the Editor through four reviewers, and the runtime stopped the decision.

```
$ algo run paper_review.algo \
    --arg "paper=Does author nationality predict citation counts?"
▶ flow main  run=eae974c4  backend=mock
  → Planner            conf=0.71  [4 items]
  → CriterionReviewer  conf=0.70  «CriterionReviewer #1393» …
  → CriterionReviewer  conf=0.75  «CriterionReviewer #9170» …
  → CriterionReviewer  conf=0.80  «CriterionReviewer #1033» …
  → CriterionReviewer  conf=0.81  «CriterionReviewer #4612» …
  → EthicsAuditor      conf=0.79  «EthicsAuditor #1139» …
✗ POLICY VIOLATION: Editor: input mentions forbidden
  attribute 'nationality'
```

Honest limit

The check is *lexical*: it blocks the words, not proxies such as surnames or home towns. A strong safeguard, not a fairness guarantee.

Two backends: learn offline, deploy live

Mock (default)

- Free, offline, no API key
- **Deterministic**: same input → same output
- Output shaped by return type
- Ideal for teaching, testing, CI

Live (`--live`)

- Real Claude models
- Role becomes the system persona
- Typed JSON reply: `{value, confidence}`
- Token budgets cap reply length

Same program, same policies, same audit log — switch with one flag: `algo run brief.algo --arg "topic=AI" --live`

04

Three ways to work



One language, three ways to use it

1 · Simple: command line

`algo` numbered menu

`algo shell` type agents line by line

`algo run f.algo`

`algo check f.algo`

For: quick runs, scripts, lab sessions

2 · Graphical

`algo studio`

AlgoLang Studio in your browser: editor, ▶ Run, results, agent graph, Python view, problems, templates

For: writing, running and *seeing* programs

3 · Agentic

`algo ask "…"`
AI writes the program; the compiler checks it; errors go back for repair

`algo wizard`
six questions, no AI needed

For: going from an idea to a program in minutes

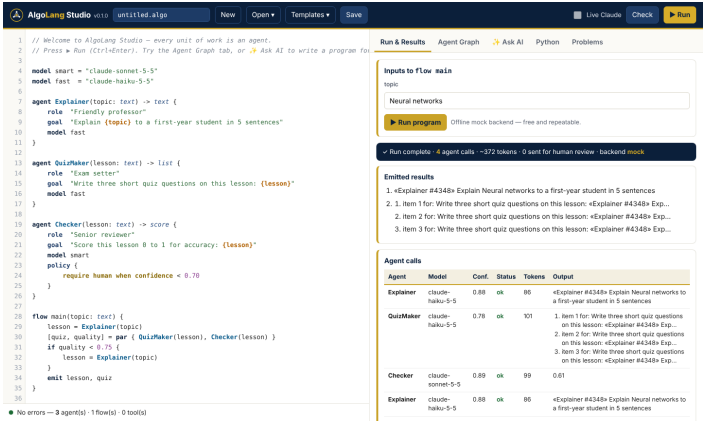
All three use the same compiler and runtime: a program written one way runs unchanged in the others.

Simple: the menu and the interactive shell

```
$ algo
  AlgoLang 0.1.0 — every unit of work is an agent
   SIMPLE (command line)
     1  Create a new program    4  Interactive shell
     2  Run a program           5  Environment check
     3  Check a program for errors
   GRAPHICAL
     6  AlgoLang Studio (opens in your browser)
     7  Jupyter notebook
   AGENTIC
     8  Ask AI to write a program from your
    ↪  description
     9  Wizard: build a program by answering
    ↪  questions
     0  Exit
  Choose 0-9:
```

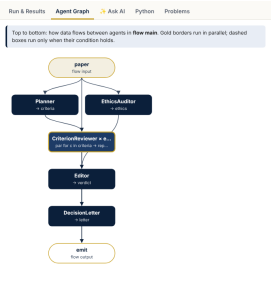
```
$ algo shell
algo> agent Explainer(topic: text) -> text {
...>     goal "Explain {topic} simply"
...> }
✓ Explainer
algo> agent Bad(x: text) -> text { goal "use {y}" }
✗ not added — 1 error(s):
  E109 line 1: goal mentions {y} but there is no parameter
 ↪  'y'
algo> Explainer("graph neural networks")
  → Explainer   conf=0.64  «Explainer #8884» Explain graph…
algo> :graph
  ◇ topic        flow input
  ■ Explainer    → lesson      ← topic
  ■ QuizMaker    → quiz        ← Explainer
algo> :save lesson.algo
✓ saved lesson.algo
```

Graphical: AlgoLang Studio

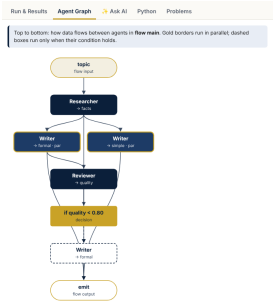


- Highlighted editor; errors marked as you type
- ▶ **Run** (Ctrl+Enter) with an input form
- Results, confidences, human-review flags
- Agent graph · Python · Problems
- Six pattern templates
- Local only; other sites blocked

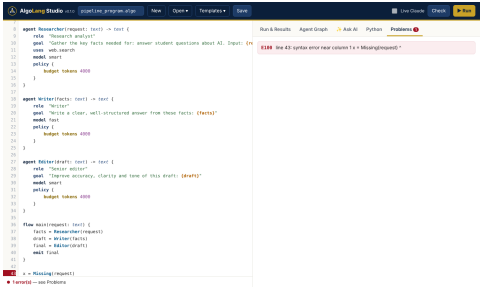
Studio: see the agents — and fix the errors



Paper review: planner, parallel reviewers, ethics auditor, editor



Research brief: parallel writers, reviewer, conditional redraft

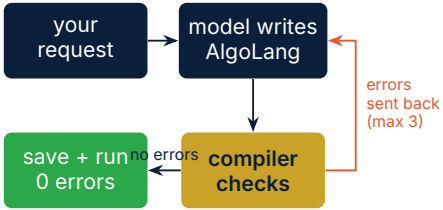


Problems tab: click an error to jump to its line, marked red in the margin.

Diagram legend

Gold border = runs in parallel · **gold diamond** = decision · **dashed** = runs only when its condition holds

Agentic: describe it — AI writes it — the compiler verifies it




```
$ algo ask "build a team that reviews a research paper"
```

Also in Studio

Ask AI tab — with undo.

```
→ attempt 1: asking claude-sonnet-5-5 to write the program…
✗ attempt 1: 2 compile error(s) — sending them back
    line 2: E109 agent Reviewer: goal mentions {manuscript}
            but there is no parameter 'manuscript'
    line 7: E101 'summarise' is not an agent
→ attempt 2: asking claude-sonnet-5-5 to write the program…
✓ attempt 2: compiles with no errors
```

Still review it

The compiler guarantees a generated program is *well-formed*. Whether its goals and thresholds are *right* is a human judgement.

Agentic without AI: the wizard

```
$ algo wizard
1. What should your AI system do: help students prepare for viva
↪  exams
2. Choose a design pattern:
   1. Pipeline       2. Fan-out / fan-in   3. Committee
   4. Critic & revise 5. Router            6. Guardrail
   Pattern number [2]: 3
3. Name of the input (one word) [request]: question
4. Send low-confidence decisions to a human? (y/n) [y]:
   Confidence threshold (0-1) [0.75]: 0.8
5. Forbid protected attributes in decisions? (y/n) [y]:
   Which ones [gender, caste, religion, nationality]:
6. Token budget per agent [4000]: 3000
✓ wrote help_students_prepare_for_viva.algo (committee
  pattern) — it compiles with no errors.
```

Six questions → a working program

Purpose · pattern · input · human review · protected attributes · budget

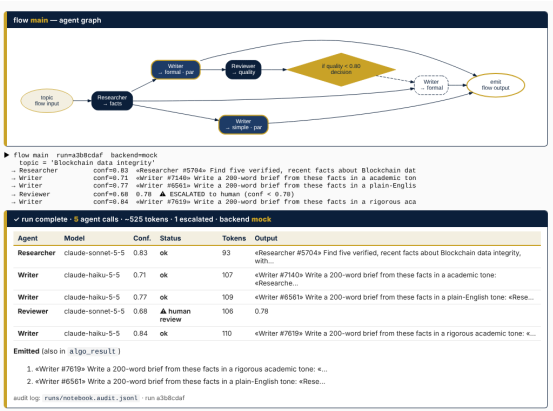
Always compiles

Generated from tested pattern templates, with governance placed on the agent that makes the final decision.

No API key needed

Ideal for classrooms and first-time users.

Jupyter and Google Colab




```
flow main(topic: text) {
    facts = Researcher(topic)
    [a, b] = par { Writer(facts),
                   Critic(facts) }
    emit a, b
}
```

- **Colab:** upload one notebook → *Run all*
- `--define` cells build a program step by step
- `--graph` · `--check` · `--python` · `--live`
- Results in `algo_result` and `algo_calls`

Runs everywhere



Windows
`double-click install.bat`

Linux · macOS
`bash install.sh`

Docker
`docker run algolang …`

Google Colab
upload notebook
→ Run all

VS Code
`syntax colours`
`Ctrl+Shift+B`

Only one requirement

Python 3.9+. The installer creates a private `.venv` and ends with `algo doctor`.

Command cheat sheet

`algo doctor` · `new` · `check` · `build` · `run` · `shell` · `studio` · `ask` · `wizard` · `notebook`

05

Inside the compiler



From .algo to governed Python



AlgoLang

```
agent A(…)

A(x, k = y)

par { A(x), B(x) }

par for v in xs {…}

if a < 0.8

emit a, b
```

Generated Python

```
A = rt.agent(AgentSpec(…))

A(p1=x, k=y)

rt.par(lambda: …)

rt.par_map(lambda v: …)

if rt.value(a) < 0.80

rt.emit(a, b)
```

```python
Researcher = rt.agent(AgentSpec(
    name='Researcher', params=[('topic', 'text', None)],
    returns='text', role='Senior research analyst',
    goal='Find five verified, recent facts about {topic}',
    uses=['web.search'], model='smart',
    policy={'budget_tokens': 6000, 'retry': 2}))

def flow_main(topic):
    facts = Researcher(topic=topic)
    formal, simple = rt.par(lambda: Writer(facts=facts),
        lambda: Writer(facts=facts, tone='plain-English'))
    quality = Reviewer(draft=formal)
    if rt.value(quality) < rt.value(0.80):
        formal = Writer(facts=facts, tone='rigorous academic')
    rt.emit(formal, simple)
```

06

Comparison with other languages



Feature comparison: what you get out of the box

Capability	Python	Python + agent frameworks	Java / C#	Rust	Mojo	AlgoLang
Agent is a language primitive	–	~	–	–	–	✓
Compile-time check of agent wiring	–	–	~	~	~	✓
Enforced per-agent tool capabilities	–	~	–	–	–	✓
Human review on low confidence	–	~	–	–	–	✓
Protected-attribute guard	–	–	–	–	–	✓
Per-agent token budget	–	~	–	–	–	✓
Automatic per-call audit trail	–	~	–	–	–	✓
Parallel agents as a keyword	~	~	~	~	~	✓
AI writer with compiler-verified repair	–	–	–	–	–	✓
Mature library ecosystem	✓	✓	✓	✓	~	–
Raw performance	~	~	✓	✓	✓	~
General-purpose computing	✓	✓	✓	✓	✓	–

✓ built in ~ possible via libraries, add-ons or hand-written code – not available

Author's assessment of built-in support, October 2026. "Agent frameworks" = e.g. LangGraph, CrewAI.

Where AlgoLang wins — and where others win

AlgoLang is stronger at

- Making AI work **visible**: every worker is declared
- Catching wiring mistakes **before** any API spend
- Governance that cannot be **forgotten** by a caller
- **Bounded** programs: maximum cost readable from source
- **Teaching** agentic AI and AI ethics with real code

Others are stronger at

- **Ecosystem**: Python's libraries, community, tooling
- **Performance**: Rust, C++, Mojo for compute-heavy work
- **General computing**: AlgoLang deliberately has no arithmetic or loops
- **Maturity**: v0.1 is a preview; tools are declared but not yet executed

AlgoLang is not a replacement for Python.
It is the **governed orchestration layer** — and it compiles *to* Python.

Same program, two languages

Plain Python — governance written by hand (excerpt of 36 lines)

```
def agent(name, role, goal, model, budget=None,
          human_below=None, forbid=(), **args):
    prompt = f"ROLE: {role}\nGOAL: " + goal.format(**args)
    for word in forbid:
        if re.search(rf"\b{word}\b", prompt, re.I):
            raise PermissionError(f"{name}: forbidden {word}")
    if budget and len(prompt) // 4 > budget:
        raise PermissionError(f"{name}: over token budget")
    out = call_llm(model, role, prompt, budget or 2000)
    status = ("escalated" if human_below and
              out["confidence"] < human_below else "ok")
    AUDIT.write(json.dumps({"agent": name, "status": status,
        "confidence": out["confidence"], "ts": time.time()}))
    return out["value"], out["confidence"]

def main(topic):
    facts, _ = agent("Researcher", "Senior research analyst",
        "Find five verified facts about {topic}",
        "claude-sonnet-5-5", budget=6000, topic=topic)
    with ThreadPoolExecutor() as pool:
        f1 = pool.submit(agent, "Writer", ..., facts=facts)
        ...
```

Similar length — different guarantees

Both versions are about 37 lines. The difference is *what is guaranteed*:

Mistake	Python	AlgoLang
Typo in `{facts}`	crash at run time	E109 before running
Call a missing agent	crash at run time	E101 before running
Forgot `forbid` on one call	silently un-guarded	policy is on the agent
Forgot audit on one call	silently miss-ing	always writ-ten
Unbounded retry loop	possible	impossible

Both programs were run; the Python version used a stub model.

The new wave of agent-era languages

Jac / Jaseci — University of Michigan

Python-like language and runtime that treat AI and AI agents as first-class entities.

Zero — Vercel Labs (May 2026)

Experimental systems language designed for AI *coding* agents: structured compiler output, explicit effects, predictable memory.

nanolang — Jordan Hubbard (2026)

Minimal, LLM-friendly language with mandatory testing and unambiguous syntax.

Mojo — Modular

Python-family language focused on high performance for AI and ML code.

AlgoLang's distinct focus: governance of multi-agent systems — human oversight, protected attributes, budgets, capabilities and audit built into the syntax, plus a teaching-first toolchain.

07

Real applications



Eight domains, eight working programs

Education
AI tutor, quiz, graded with oversight

Finance & scholarships
fair screening, separate auditor

Customer support
route each ticket to a specialist

Agriculture
crop advice in Hindi, Marathi, Telugu

Healthcare
clinical-note triage with a human gate

Cybersecurity
parallel log triage, incident report

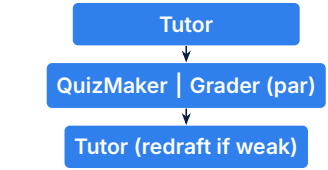
Legal
contract clause risk review

Research
literature review for Ph.D. scholars

Every program on the next slides is printed from its real file in `examples/apps/`, and every one compiles and runs.

Education: an AI tutor that explains, quizzes and grades

```
// Education — AI tutor that explains, quizzes and grades
model fast = "claude-haiku-5-5"
agent Tutor(topic: text, level: text = "first-year") -> text {
    goal "Explain {topic} for a {level} student in 6 sentences"
    model fast
}
agent QuizMaker(lesson: text) -> list {
    goal "Write 5 exam questions on: {lesson}"
}
agent Grader(lesson: text) -> score {
    goal "Score the lesson 0 to 1 for accuracy: {lesson}"
    policy { require human when confidence < 0.80 }
}
flow main(topic: text) {
    lesson = Tutor(topic)
    [quiz, quality] = par { QuizMaker(lesson), Grader(lesson) }
    if quality < 0.75 { lesson = Tutor(topic, level = "beginner") }
    emit lesson, quiz
}
```



Impact

Personalised lessons for every level; a grader's uncertain marks go to the teacher, not the student.

Healthcare: clinical-note triage with a human gate

```
// Healthcare — clinical note triage with a mandatory human gate
model smart = "claude-sonnet-5-5"
agent Summariser(note: text) -> text {
    goal "Summarise symptoms and history in this note: {note}"
}
agent RiskScorer(summary: text) -> score {
    role "Clinical decision-support assistant"
    goal "Score urgency 0 to 1 from: {summary}"
    model smart
    policy {
        require human when confidence < 0.95
        forbid religion, caste
    }
}
agent Referral(summary: text) -> text {
    goal "Draft a referral note for the doctor from: {summary}"
}
flow main(note: text) {
    summary = Summariser(note)
    urgency = RiskScorer(summary)
    if urgency >= 0.7 { emit Referral(summary) }
    emit summary, urgency
}
```

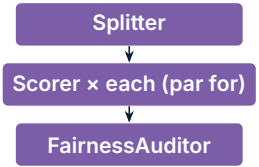

human < 0.95 · forbid · branch

Impact

Faster triage, with a very high review threshold: the doctor decides. Decision support, never diagnosis.

Finance & scholarships: fair screening

```
// Finance / scholarships — fair screening with separation of duties
agent Splitter(batch: text) -> list {
    goal "List each application in {batch} separately"
}
agent Scorer(application: text) -> score {
    goal "Score {application} on the published merit criteria only"
    policy {
        forbid gender, caste, religion, nationality
        require human when confidence < 0.85
        budget tokens 3000
    }
}
agent FairnessAuditor(scores: list) -> text {
    role "Independent fairness auditor"
    goal "Check {scores} for unfair patterns across groups"
}
flow main(batch: text) {
    apps   = Splitter(batch)
    scores = par for a in apps { Scorer(a) }
    audit  = FairnessAuditor(scores)
    emit scores, audit
}
```



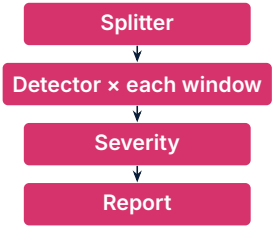
forbid 4 attributes | human | budget

Impact

Merit-only scoring; an *independent* auditor checks outcomes across groups — separation of duties.

Cybersecurity: parallel log triage

```
// Cybersecurity — parallel log triage and incident report
tool file.read
agent Splitter(logs: text) -> list {
    goal "Split {logs} into time windows for analysis"
    uses file.read
}
agent Detector(window: text) -> text {
    role "SOC analyst"
    goal "Find anomalies and indicators of compromise in {window}"
    policy { budget tokens 5000  retry 2 }
}
agent Severity(findings: list) -> score {
    goal "Score overall severity 0 to 1 of {findings}"
    policy { require human when confidence < 0.80 }
}
agent Report(findings: list, sev: score) -> text {
    goal "Write an incident report (severity {sev}) from {findings}"
}
flow main(logs: text) {
    windows  = Splitter(logs)
    findings = par for w in windows { Detector(w) }
    sev      = Severity(findings)
    emit Report(findings, sev)
}
```



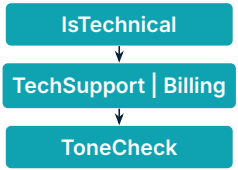
par for · budget + retry · human

Impact

Many log windows analysed at once; a SOC analyst reviews uncertain severity before escalation.

Customer support: route every ticket

```
// Customer support — route each ticket to the right specialist
agent IsTechnical(ticket: text) -> score {
    goal "Score 0 to 1 how technical this ticket is: {ticket}"
}
agent TechSupport(ticket: text) -> text {
    goal "Give step-by-step technical help for: {ticket}"
}
agent Billing(ticket: text) -> text {
    goal "Resolve this billing or account question: {ticket}"
}
agent ToneCheck(reply: text) -> score {
    goal "Score 0 to 1 how polite and clear this reply is: {reply}"
}
flow main(ticket: text) {
    if IsTechnical(ticket) >= 0.5 {
        reply = TechSupport(ticket)
    } else {
        reply = Billing(ticket)
    }
    if ToneCheck(reply) < 0.8 { reply = Billing(ticket) }
    emit reply
}
```



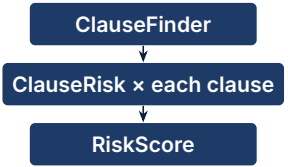
router quality gate

Impact

Each ticket reaches the right specialist; replies are checked for tone before they go out.

Legal: contract clause risk review

```
// Legal — contract clause review with lawyer sign-off
tool file.read
agent ClauseFinder(contract: text) -> list {
    goal "List every clause in {contract}"
    uses file.read
}
agent ClauseRisk(clause: text) -> text {
    role "Contract analyst"
    goal "Flag risks and missing protections in: {clause}"
}
agent RiskScore(notes: list) -> score {
    goal "Score overall contract risk 0 to 1 from {notes}"
    policy { require human when confidence < 0.90 }
}
flow main(contract: text) {
    clauses = ClauseFinder(contract)
    notes   = par for c in clauses { ClauseRisk(c) }
    emit RiskScore(notes), notes
}
```



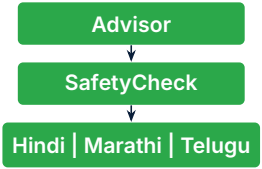
par for · human < 0.90 · file.read

Impact

Every clause reviewed in parallel; the lawyer signs off whenever the model is not highly confident.

Agriculture: crop advice in three Indian languages

```
// Agriculture — crop advisory in three Indian languages
tool web.search
agent Advisor(crop: text, region: text) -> text {
    goal "Give weather-aware sowing and pest advice for {crop} in {
      region}"
    uses web.search
}
agent Translator(advice: text, language: text) -> text {
    goal "Translate for farmers into {language}, keep it simple: {
      advice}"
}
agent SafetyCheck(advice: text) -> bool {
    goal "Answer true only if {advice} contains no unsafe pesticide
      dosing"
}
flow main(crop: text, region: text) {
    advice = Advisor(crop, region)
    if SafetyCheck(advice) == true {
        [hi, mr, te] = par {
            Translator(advice, language = "Hindi"),
            Translator(advice, language = "Marathi"),
            Translator(advice, language = "Telugu")
        }
        emit advice, hi, mr, te
    }
}
```



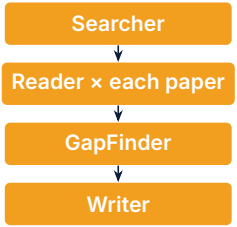
guardrail · par translate · web.search

Impact

Weather-aware advice reaches farmers in their own language — only after a pesticide-safety check.

Research: literature review for Ph.D. scholars

```
// Research — literature review for a Ph.D. scholar
tool web.search
agent Searcher(question: text) -> list {
    goal "Find 6 key recent papers on {question}"
    uses web.search
}
agent Reader(paper: text, question: text) -> text {
    goal "Summarise method, data and findings of {paper} for {question}
      "
}
agent GapFinder(summaries: list) -> text {
    goal "Identify open research gaps across {summaries}"
}
agent Writer(summaries: list, gaps: text) -> text {
    goal "Write a related-work section from {summaries}, ending with {
      gaps}"
}
flow main(question: text) {
    papers    = Searcher(question)
    summaries = par for p in papers { Reader(p, question) }
    gaps      = GapFinder(summaries)
    emit Writer(summaries, gaps)
}
```

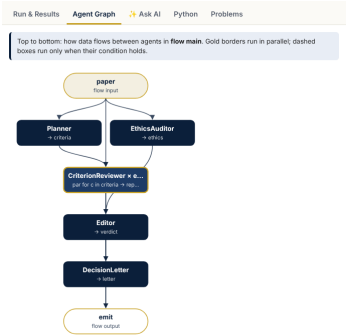


fan-out / fan-in web.search

Impact

Six papers read in parallel, gaps identified, a related-work draft produced — for the scholar to verify.

Also built in: academic peer review



paper_review.algo — an editorial office in 50 lines

1. **Planner** picks the review criteria
2. **CriterionReviewer** runs once per criterion, in parallel
3. **EthicsAuditor** checks consent, bias, reproducibility
4. **Editor** scores — blind to author identity (`forbid`)
5. **DecisionLetter** drafts a courteous letter

Who it is for

Journal editors, conference chairs and Ph.D. supervisors who want faster, fairer, auditable first-round reviews.

08

Responsible AI & teaching



Eight multi-agent design patterns

Pipeline
Stages with different skills

Fan-out / fan-in
Speed on decomposable work

Committee
Several experts, one judge

Critic & revise
Bounded rounds of improvement

Router
Right specialist per input

Guardrail
Safety check before work

Human-in-the-loop
Gate the decisions that matter

Separation of duties
Decider ≠ auditor

The first six are available as one-click templates in Studio and in `algo wizard`.

Responsible AI: principles mapped to features

Principle	How AlgoLang supports it	Feature
Accountability	Every unit of work is named and has a goal; every call recorded	`agents only, audit log`
Human oversight	Low-confidence results routed to people; reviewers approve	`require human when`
Fairness	Protected attributes rejected as inputs and blocked in prompts	`forbid (E108)`
Transparency	Capabilities, models, policies visible in source and logs	`tool, uses, model`
Security	Agents receive only explicitly granted capabilities	`declared tools (E104)`
Privacy	Log stores prompt hashes, not prompts or outputs	`inputs_sha`
Sustainability & cost	Per-agent budgets; no unbounded loops	`budget tokens`
Reliability	Retries, timeouts, deterministic offline testing	`retry, timeout, mock`

Produces evidence useful for frameworks such as the NIST AI RMF and the EU AI Act's record-keeping and human-oversight provisions — but no language alone makes a system compliant.

Known limitations of version 0.1

forbid is lexical

Blocks the words, not proxies (surname, town, school).

Confidence is self-reported

Not a calibrated probability; validate before trusting thresholds.

Tools are declared, not executed

Executable tool bindings (incl. MCP) planned for v0.3.

Argument types not yet checked

Planned for v0.2; flow inputs arrive as text.

Prompt injection

Policies limit what agents can do, but do not detect injected instructions.

Audit log is a local file

Not tamper-proof — protect and back it up.

Not for sole use in legally significant decisions about people without independent validation and human review.

Teaching with AlgoLang: a 10-lab module

Lab	Topic	Students build	Concepts
1	First agents	The starter program; change goals and roles	agent, goal, flow
2	Types and scores	A grader that returns a score, and a branch	types, if, E110
3	Reading errors	Fix a program with eight errors	diagnostics
4	Parallel agents	A committee of three experts and a judge	par
5	Planning	A planner and par for workers	fan-out / fan-in
6	Capabilities	Grant and remove tools	least privilege
7	Human oversight	Gate a decision agent; –interactive	thresholds
8	Fairness	Screening with forbid; probe with proxies	fairness limits
9	Audit	Load the audit log into pandas	accountability
10	Going live	Run a finished flow on Claude with budgets	cost, real output

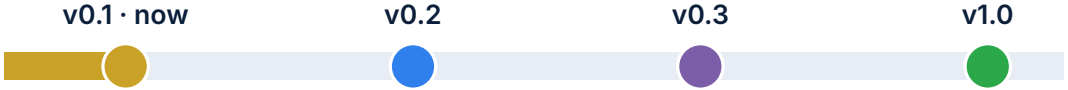
Free and repeatable on the mock backend; zero installation in Google Colab.

09

The open-source project



Roadmap



v0.1 · now

Agents, flows, policies
par / par for
mock + live backends
audit log · notebooks
Studio · shell · ask · wizard

v0.2

Argument type checking
`uncertain<T>` type
`eval` blocks with trace replay
language server

v0.3

Executable tools (MCP bindings)
cost & energy budgets
consent-scoped memory

v1.0

Stable language
typed agent messaging
native runtime
package registry

Semantic versioning

From 1.0, programs that compile under 1.*x* compile under every later 1.*y*.

AlgoLang Enhancement Proposals

Every proposal must include a section on its responsible-AI implications.

Open source: licence, contributions, citation

Licence

Software: **Apache 2.0** · Documentation: **CC BY 4.0**

Contribute

Report problems with the `.algo` file and `algo doctor` output · add examples, teaching material, translations · keep the compiler small and readable

Community

Open to everyone · respectful and constructive · students welcome

How to cite

```
@manual{satyanarayana2026algolang,
  title  = {The {AlgoLang} Programming Language:
            Official Documentation, Version 0.1},
  author = {Satyanarayana, S.},
  organization = {AlgoProfessor AI R\&D Solutions},
  address = {Hyderabad, India},
  year   = {2026},
  url    = {https://www.algoprofessor.com}
}
```

Get started in 60 seconds

```
$ bash install.sh     # Windows: install.bat
$ algo doctor
$ algo new mylesson
$ algo run mylesson.algo --arg "topic=AI"
$ algo studio         # graphical IDE
$ algo wizard         # no-AI builder
$ algo ask "build a peer-review team"
```

In the box

Compiler · runtime · shell · Studio · AI writer · wizard · Jupyter/Colab notebook · VS Code support · Docker · 11 example programs

Read more

77-page official documentation (LaTeX + PDF): language reference, runtime, patterns, responsible AI, 10 labs

Zero install

Upload `AlgoLang_Notebook.ipynb` to Google Colab → *Run all*



Thank you

Every unit of work is an **agent**.

Dr. S. Satyanarayana, Ph.D., PDF (AI)

Founder, CEO & Chief Agentic AI Scientist · AlgoProfessor AI R&D Solutions, Hyderabad

ceo@algoprofessor.com · www.algoprofessor.com · +91-6303432833